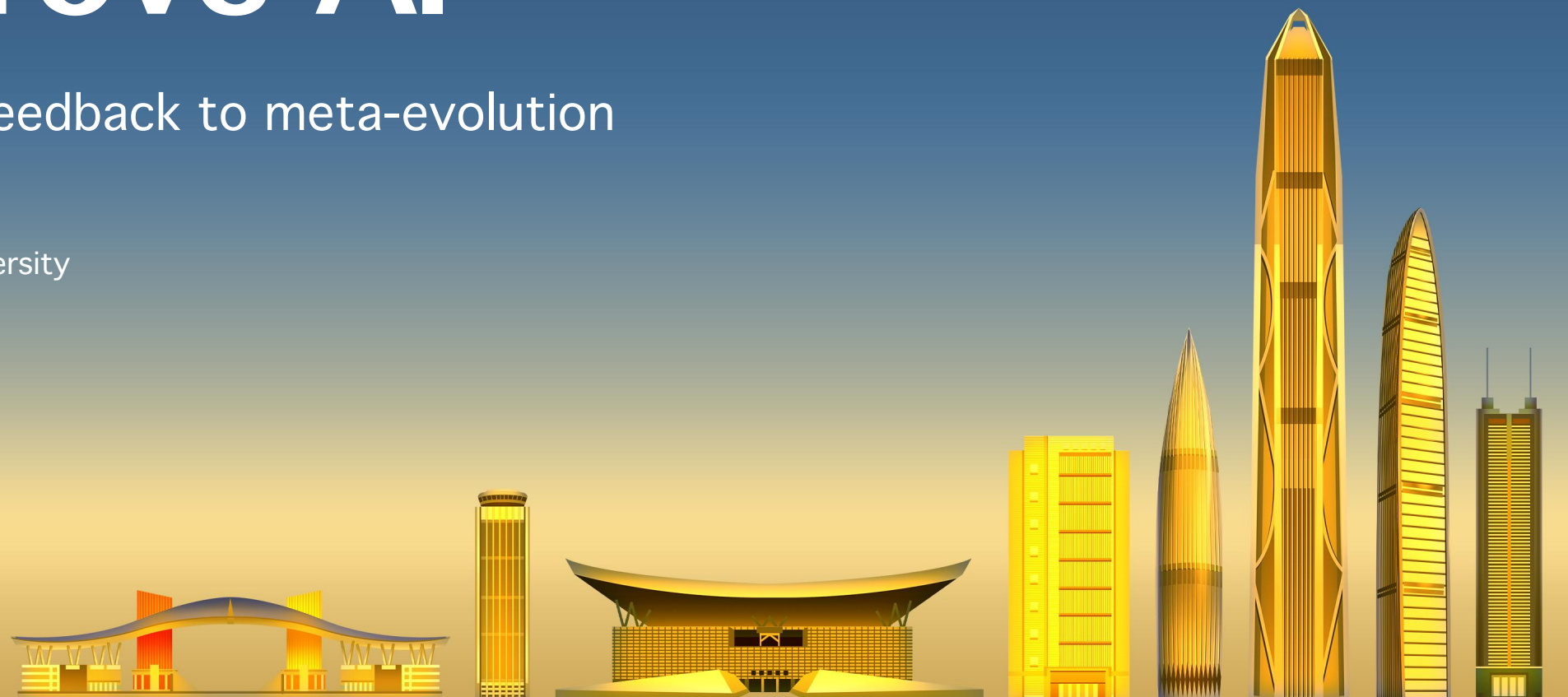
GOSIM

GOSIM Shenzhen 2026

Teaching AI to Improve AI

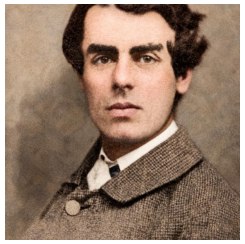
From experiment feedback to meta-evolution

Che Jiang
Research Scientist, FrontisAI
PhD Student, Tsinghua University



An old ambition now has an experimental interface

For 160 years the question was speculative. Now agents edit, run and evaluate code, so parts of it can be tested.



1863

Samuel Butler

Darwin among the Machines

“we are ourselves creating our own successors”

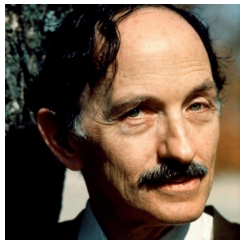


1951

Alan Turing

Intelligent Machinery, a Heretical Theory

“converse with each other to sharpen their wits”

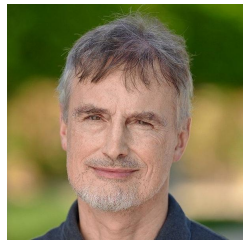


1965

I. J. Good

The First Ultraintelligent Machine

“could design even better machines”

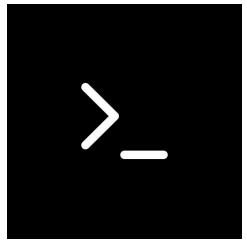


2003

Jürgen Schmidhuber

Gödel Machine

Rewrites its own code once it can prove the rewrite is useful



Now

Agents in the loop

AlphaEvolve

autoresearch · DGM · AIDE²

Propose, execute, evaluate, keep or revert

Proofs are rarely available. Experiments are, and they can be run at scale.

Butler, Darwin among the Machines (1863); Turing, Intelligent Machinery, a Heretical Theory (1951); Good (1965); Schmidhuber, Gödel Machines (2003).

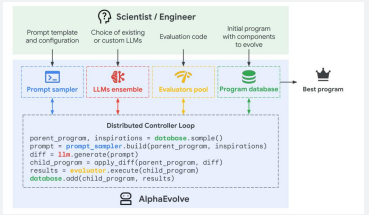
AI already runs parts of the AI research loop

Four public systems share one loop. What differs is the object being improved, and it keeps moving up.

SHARED LOOP → Propose a change → Execute → Evaluate → Keep or revert

01 ALGORITHMS & INFRA

AlphaEvolve

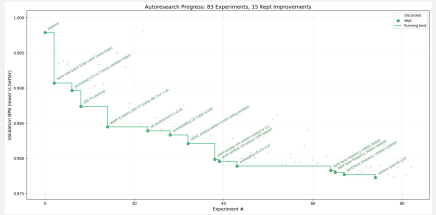


LLM-proposed program diffs, scored by automated evaluators; winners seed later prompts.

- **0.7% fleet compute recovered · 23% faster kernel**

02 A TRAINING RUN

Karpathy / Autoresearch

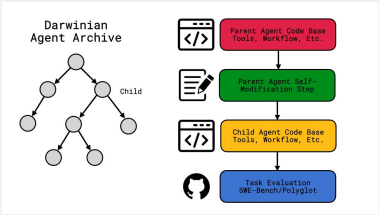


Edit train.py, train for five minutes, keep the change only if val bits-per-byte drops.

- **83 experiments, 15 kept changes**

03 A CODING AGENT

Darwin Gödel Machine

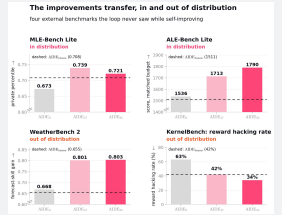


The agent rewrites its own code and tools; children join an open-ended archive.

- **SWE-bench 20.0% → 50.0%**

04 A RESEARCH AGENT

AIDE²



An outer loop rewrites the inner research agent's harness, then tests transfer.

- **Gains on 4 benchmarks never seen in the loop**

Sources: Google DeepMind, AlphaEvolve (2025); Karpathy, autoresearch (2026); Zhang et al., Darwin Gödel Machine (2025); Weco AI, AIDE² (2026).

CONTENTS

GOSIM Shenzhen 2026

01 • **Experiments become experience**
What should an AI keep from an experiment?

Survey
Self-Improving Agents in the Era of Experience · 2026

02 • **Scientific methods under test**
Can coding agents produce competitive scientific methods?

NatureBench
90 tasks from Nature-family papers · arXiv 2606.24530

03 • **Training the Improver**
Can training make a model a better improver?

Frontis-MA1 · OpenMLE
Open model, data and search stack · arXiv 2607.28568

04 • **Self-Evolution of the Improver**
Can improvement experience transfer to unseen tasks?

MetaEvoForge
Meta-evolving agents across 62 task pools · new in this talk

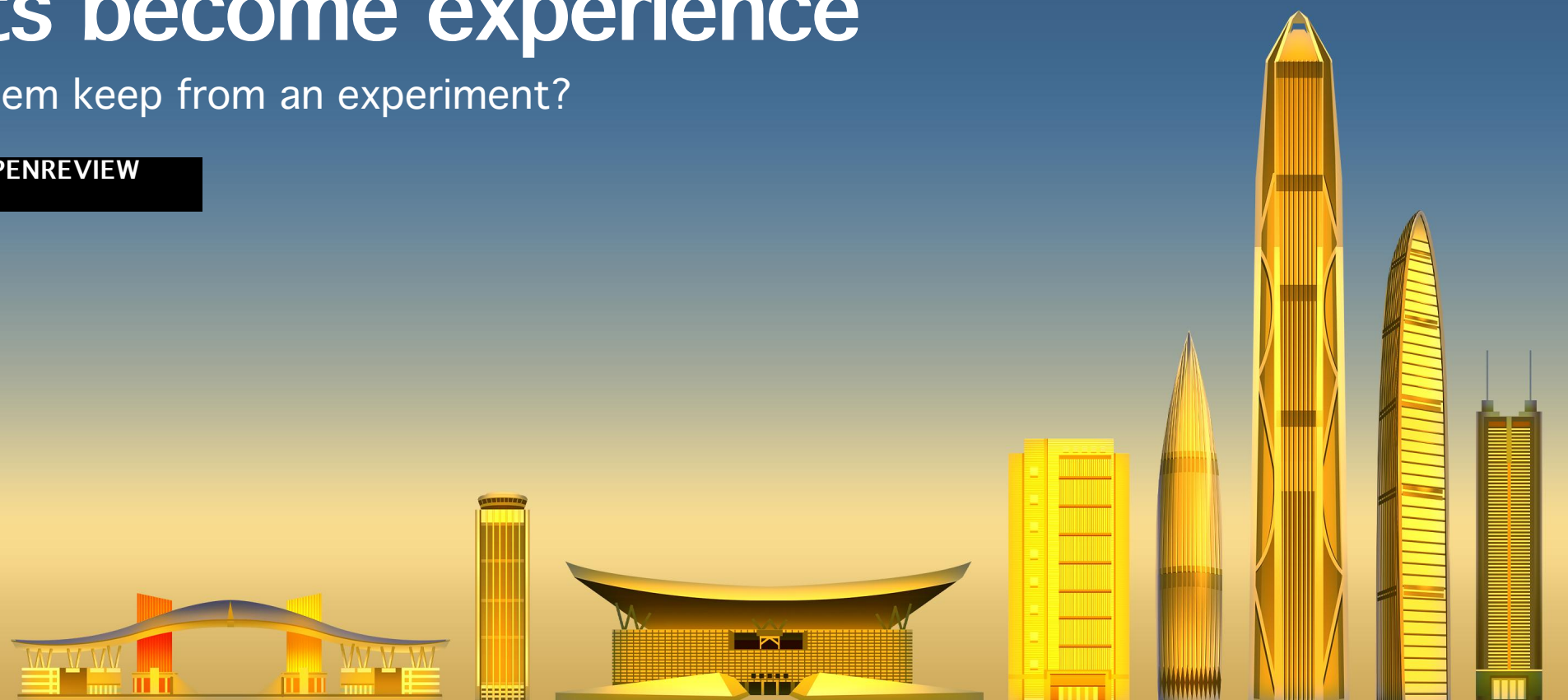


01

Experiments become experience

What should an AI system keep from an experiment?

FRAMEWORK · OUR SURVEY, OPENREVIEW
2026



Execution turns a guess into reusable experience

A hypothetical debugging episode. What is worth keeping is what changed, under which conditions, and why it worked.

Propose
Method A

→

Run
Data-shape error

→

Revise
Method B repairs the error

→

Validate
Check an independent split

REUSABLE INTERVENTION RECORD

Conditions
Task, data and the failure that was observed

+

Change
The exact edit: method A → method B

+

Observed outcome
Measured on an independent split, not the training batch

↳ read by the next proposal

> **A conditional record carries more learning value than a score alone.**

Three routes differ in what gets better

They can coexist in one system. Adding another loop does not, by itself, move a system up the ladder.

A better system is not necessarily a better improver.

EVOLUTION

A better solution

Search over candidates for the current task. Gains end with the campaign.

AlphaEvolve · OpenMLE-Evo

SELF-EVOLUTION

A better agent

Experience is written into memory, skills, tools or weights and changes future behavior.

Voyager · SkillRL · Frontis-MA1

META-EVOLUTION

A better improver

Feedback revises how changes are proposed, selected and validated.

Hyperagents · MetaEvoForge

RSI · OPEN QUESTION

Compounding improvers

Each upgraded improver takes over the next round and keeps producing stronger successors at a fixed budget.

Might this be a way to RSI?

Source: Jiang et al., Self-Improving Agents in the Era of Experience, OpenReview (2026).

Experience can live in context, harness or weights

Each surface trades adaptation speed against durability and reach. This talk uses all three.

FAST · LOCAL ⟷ DURABLE · BROAD

Context & memory

Search state, retrieved failures and operator-specific notes, assembled per call.

- Updates every step
- Scope: the current task
- Easy to discard

In this talk **OPENMLE-EVO**

Harness & skills

Prompts, skills, tools and workflow code that persist across tasks.

- Updates per accepted edit
- Scope: future tasks
- Inspectable and revertible

In this talk **METAEVOFORGE**

Model weights

SFT and RL on execution-grounded trajectories.

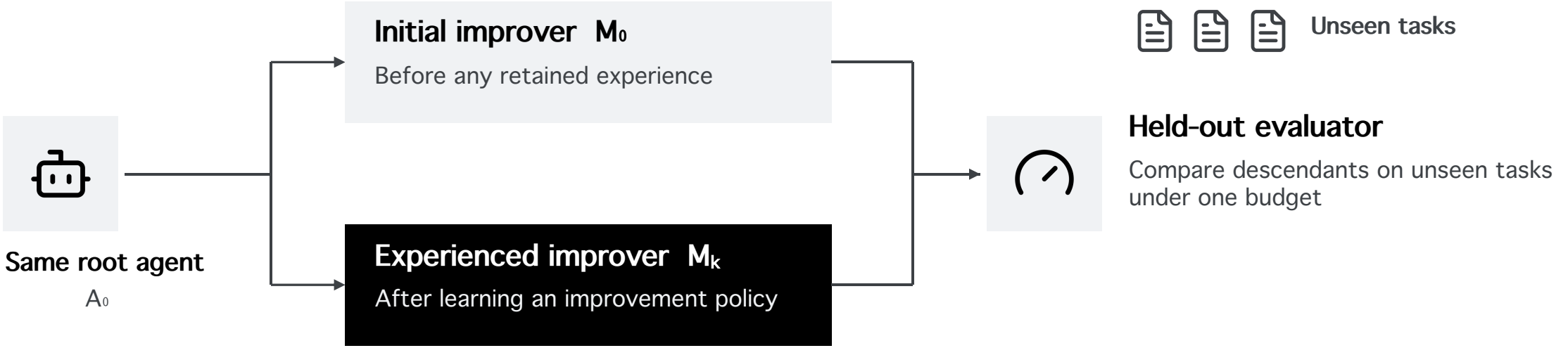
- Updates per training run
- Scope: every task the model touches
- Hard to inspect or undo

In this talk **FRONTIS-MA1**

Source: Jiang et al., Self-Improving Agents in the Era of Experience, OpenReview (2026).

A better improver must win on tasks it never saw

Hold the starting agent and the budget fixed. Vary only the improver's experience.



EVIDENCE THAT COUNTS

- A stronger final agent at one budget
- The same target with fewer resources
- Failures that show where transfer stops

Improvement needs a credible test

Experience matters when it improves a new task under a comparable budget.

WHAT WE KEEP

Conditions

Task, data and the observed failure

Change

The exact intervention

Outcome

Measured on an independent split

NEXT: SCIENTIFIC METHODS UNDER TEST

Can agents match published scientific methods?

NatureBench

Real data, executable tasks and a published result as the reference.

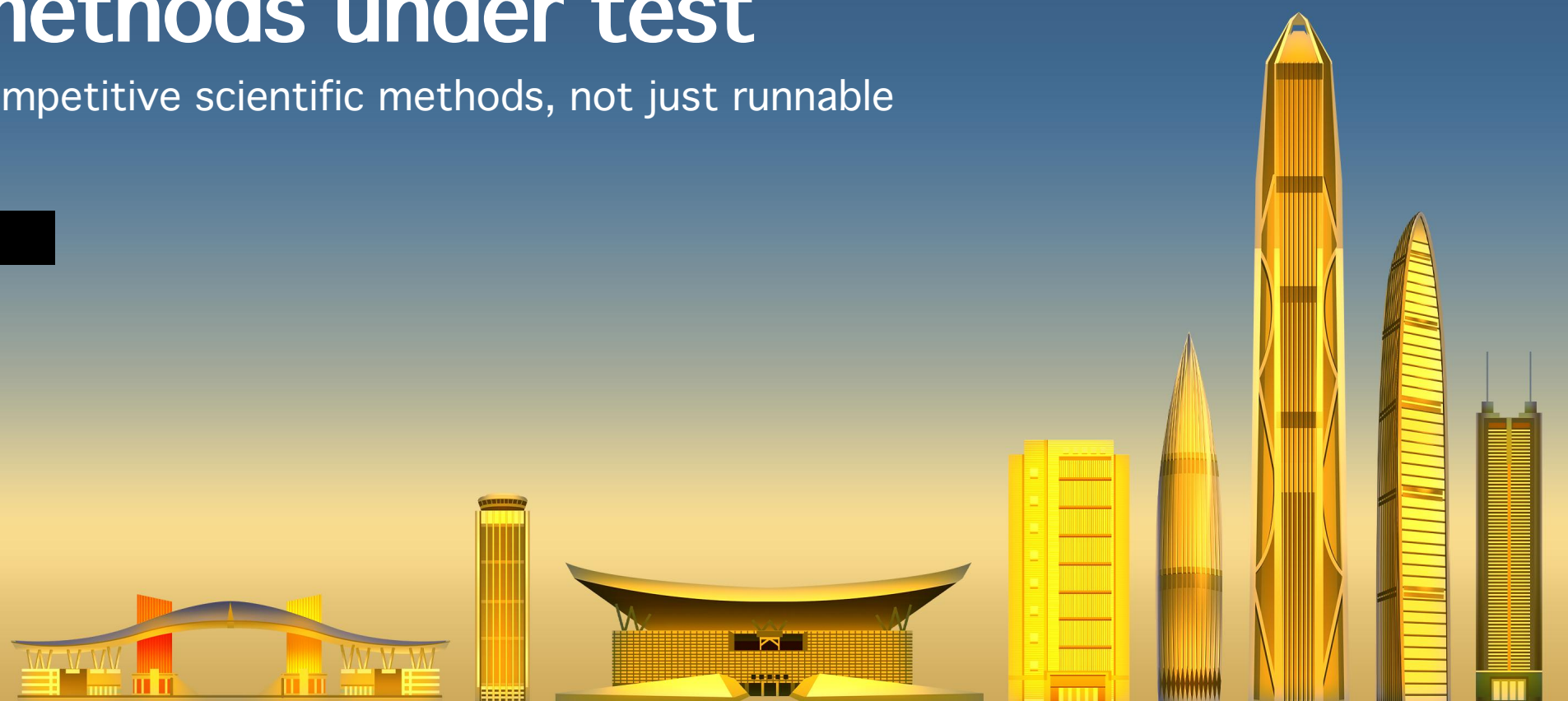
A benchmark turns an improvement claim into an experiment.

02

Scientific methods under test

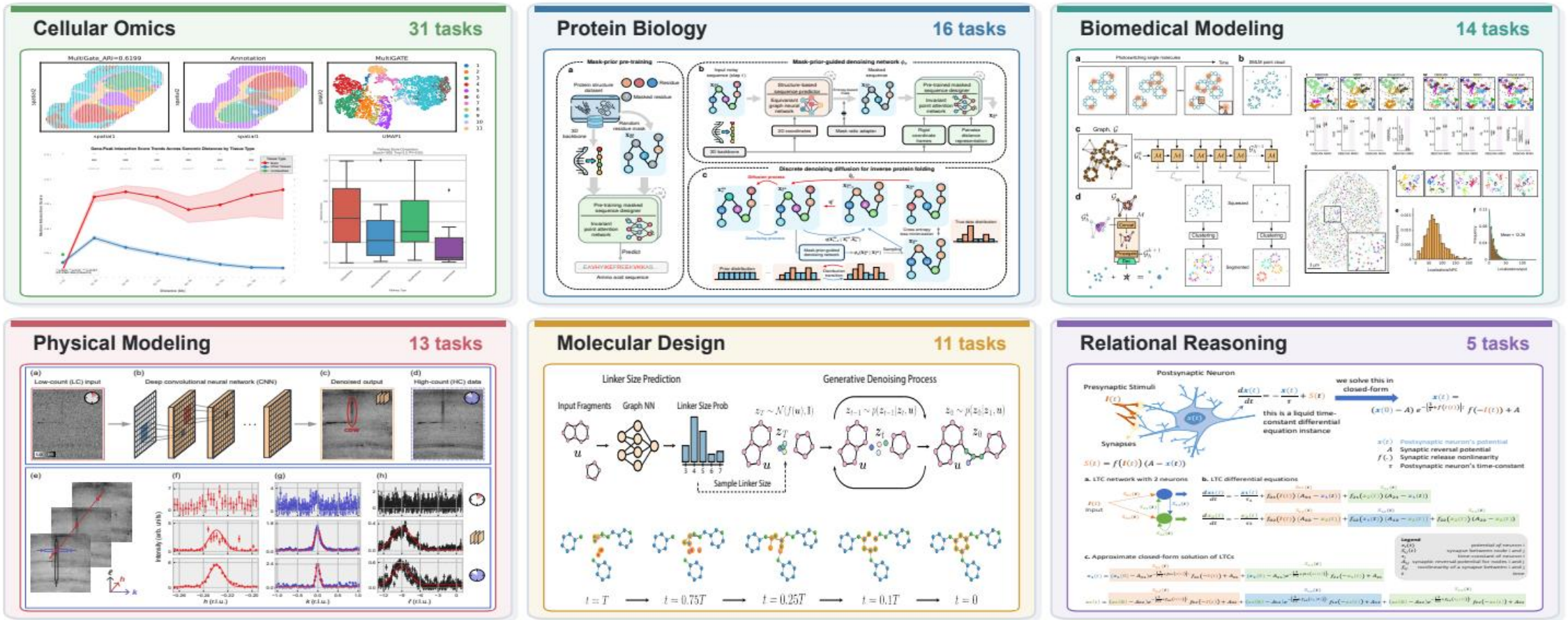
Can agents produce competitive scientific methods, not just runnable code?

NATUREBENCH · ARXIV 2606.24530



NatureBench: 90 tasks from Nature-family papers

333 task instances across 6 domains · Real data, runnable environments and published baselines



Source: Wang et al., NatureBench, arXiv:2606.24530, Fig. 3 (2026).

NatureGym turns a paper into an executable experiment

Keep the scientific problem; withhold the paper's method and the test answers. Quality gates decide what survives.

Research paper
Task definition and source data

→

Information boundary
Remove the method and hidden labels

→

Runnable package
Public inputs + sandbox contract

→

Your agent
Proposes and runs a method

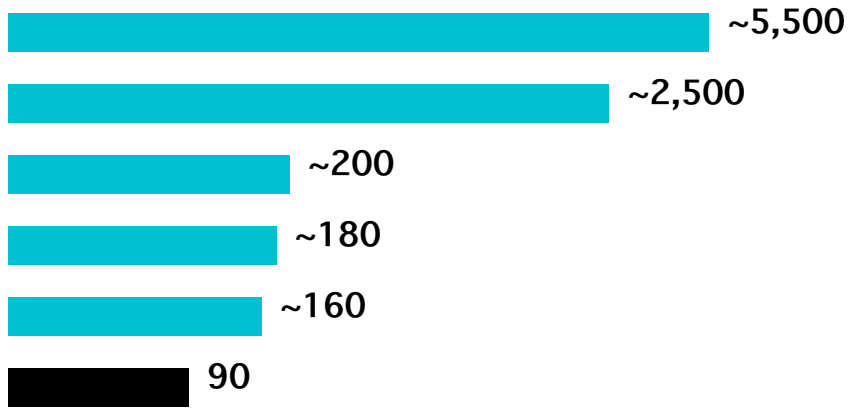
→

Hidden evaluator
Labels + paper-level anchor

~5,500 → 90

Articles in, calibrated tasks out. Each gate removes tasks that cannot be extracted, evaluated or reproduced.

Collection	Nature-family journals
Article filter	Research papers only
Three-level filter	Extractable and evaluable
Acquisition	Data obtainable and verifiable
Construction	Package runs end to end
Calibration	Quality-checked in real runs

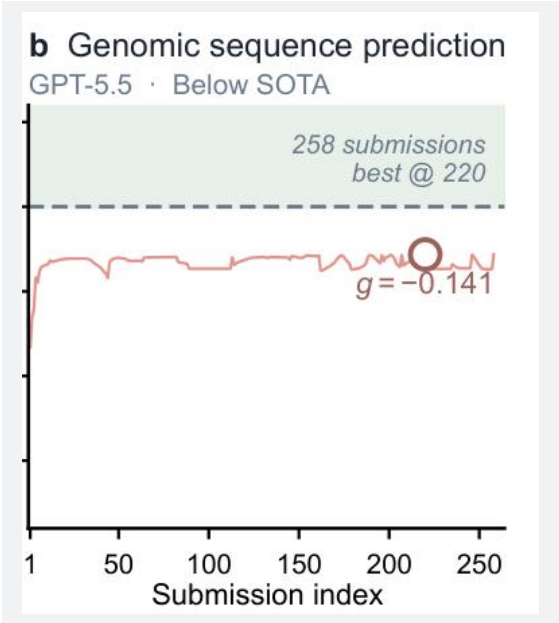


Source: Wang et al., NatureBench, arXiv:2606.24530, Table 3 (2026).

A higher score still needs a method explanation

Search traces show where local refinement helps and where it reaches a limit.

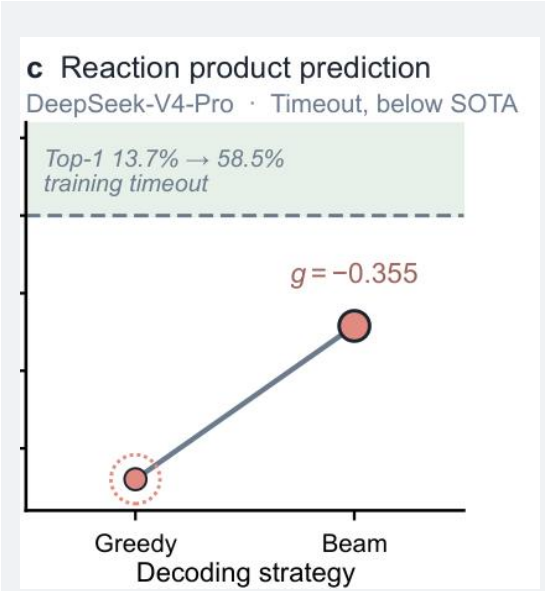
GENOMIC SEQUENCE PREDICTION



Many similar variants leave a plateau

258 submissions, best at #220, still below the paper's result.

REACTION PRODUCT PREDICTION



One decoding change moves the result

Greedy → beam search lifts top-1 from 13.7% to 58.5%, then training times out.

The metric shows that something changed; the program and its evidence show what.

NatureBench is already a shared community testbed

Bring your own agent runner, keep the artifacts, and compare through one reproduction process.

Your agent runner
Harbor adapter or the task interface

NatureBench
90 tasks · NatureBench-25 entry track

Retained artifacts
Programs, logs and raw results

External evaluations
AIBuildAI · HELIX · Luria

github.com/FrontisAI/NatureBench

NEXT · NATUREBENCH V2

Explore broader scientific problems and validate results with evidence.

In development; computational exploration and validation.

Broader scientific scope
Explore methods, designs and configurations

Scientific goals
Preserve the core problem and required validation

Results and evidence
Submit code, data, models or reports

Combined evaluation
Program execution, rules and LLM assessment

Source: NatureBench, Harbor adapter and public evaluation reports.

Evaluation tells us what to train

NatureBench measures scientific capability. OpenMLE supplies experience for training.

WHAT NATUREBENCH PROVIDES

Scientific tasks
Real data and runnable environments

A meaningful reference
The source paper's reported result

Evidence to inspect
Programs, traces and method choices

NEXT: TRAINING THE IMPROVER

Can execution experience teach better revisions?

Frontis-MA1 + OpenMLE
Train on executable transformations and test transfer on held-out tasks.

The next step is learning a reusable improvement skill.

03

Training the Improver

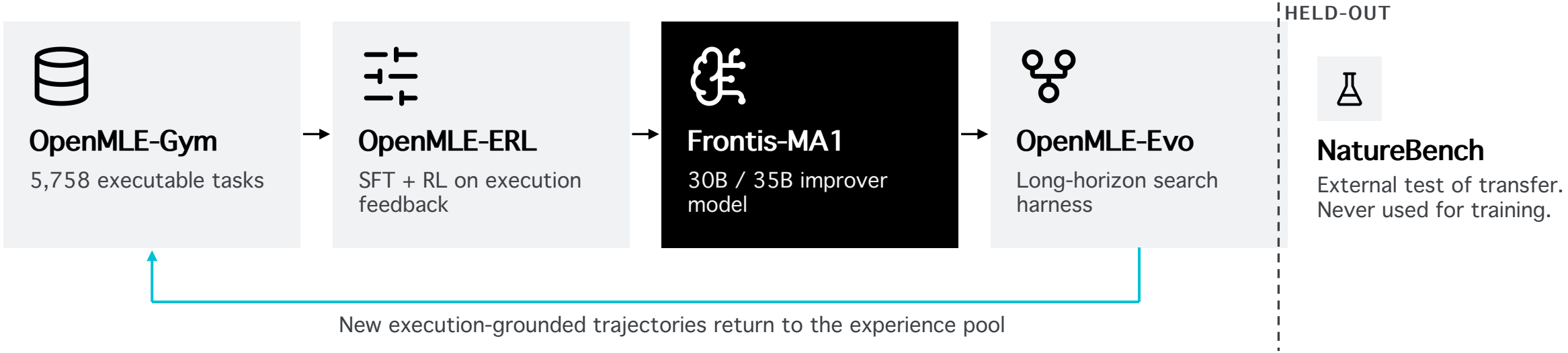
Can training make a model a better improver, not just a better solver?

FRONTIS-MA1 · OPENMLE · ARXIV 2607.28568



One execution loop links tasks, training and search

Gym supplies experience, ERL trains the improver, Evo composes it at test time. NatureBench stays held out.



The model learns local transformations; the harness decides how to compose them.

Source: Yang et al., Frontis-MA1: Training an AI4AI Model towards RSI in MLE, arXiv:2607.28568 (2026).

5,758 executable environments behind one interface

Three task sources enter one verified package contract. The sandbox returns structured outcomes, not just a score.

Curated anchors
156

Kaggle datasets
3,362

Kaggle competitions
2,240

Quality gates
Execution checks + semantic validation

5,758 environments

ONE PACKAGE CONTRACT

```
task_package/
  public/
    description.txt
    data inputs
  private/
    hidden labels
  utils/
    prepare.py
    metric.py
```

SANDBOX OUTCOMES

- Success, with a score
- Runtime error
- Missing code or submission
- Scoring failure
- Timeout

A crash needs a different revision than a weak score.

Four transformations connect training and search

SFT and RL teach the same operators that the search harness calls at test time.

TRAINING 26,259 SFT examples (17,245 full solutions + 9,014 trajectory steps) + RL on execution feedback



Draft

Create a runnable candidate from the task spec



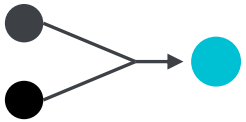
Improve

Refine a valid parent using its feedback



Debug

Repair a failing parent using its errors



Crossover

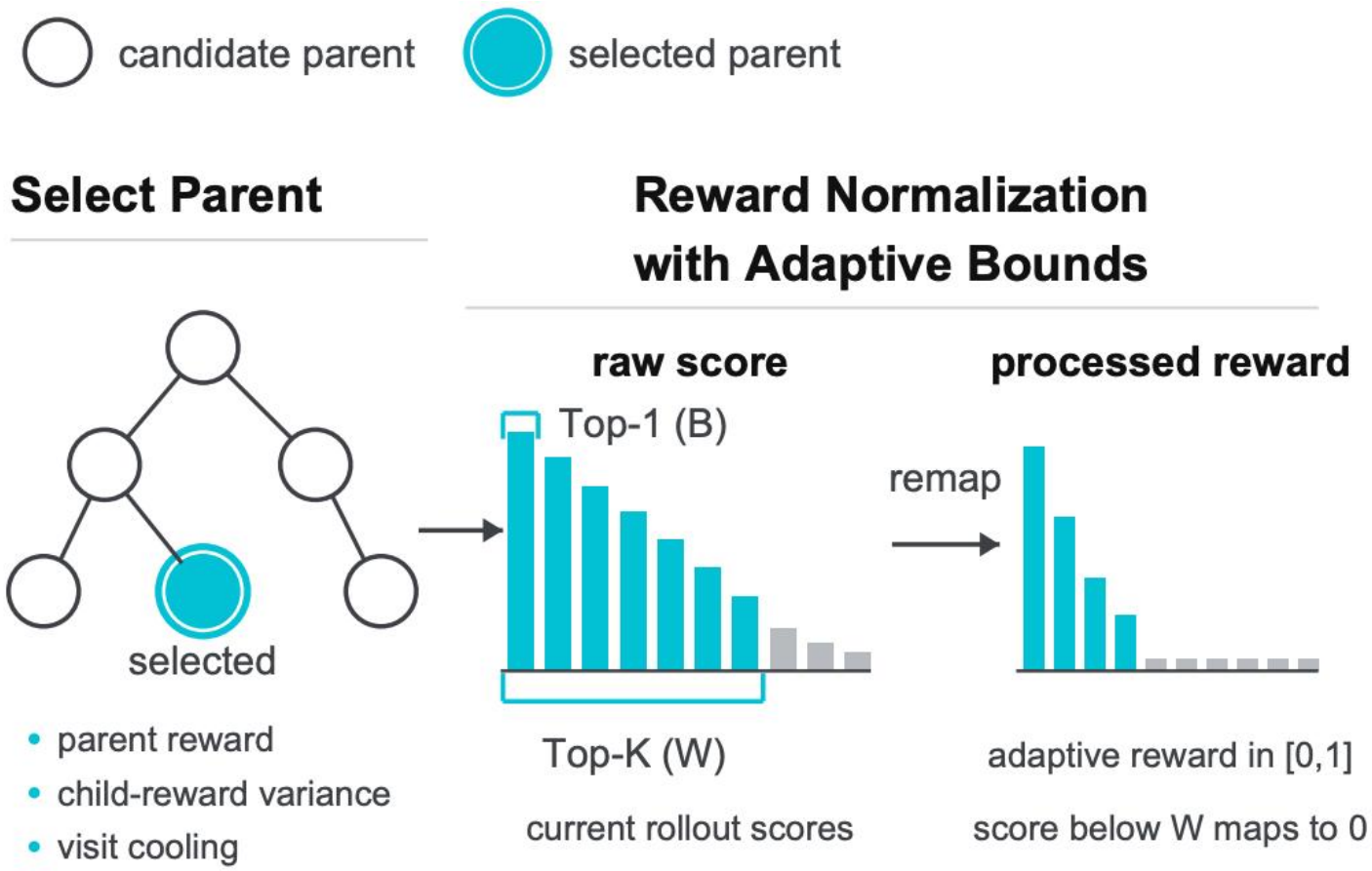
Combine two complementary parents

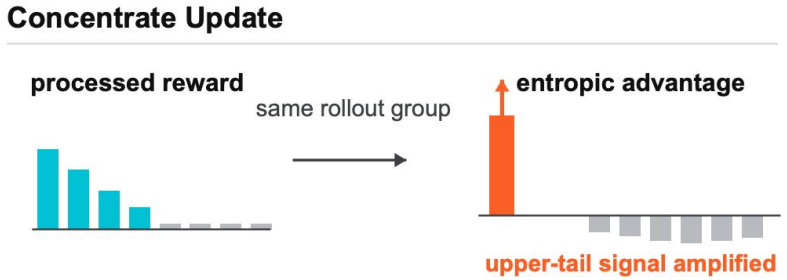
SEARCH choose a parent → call an operator → store the new evidence

Source: Yang et al., Frontis-MA1, arXiv:2607.28568, Figs. 5 and 7 (2026).

Execution feedback becomes a calibrated reward

Task-adaptive normalization preserves real gains; entropic advantages emphasize stronger candidates.





Improvement bonus

Reward revisions that beat their parent.

Asynchronous rollouts

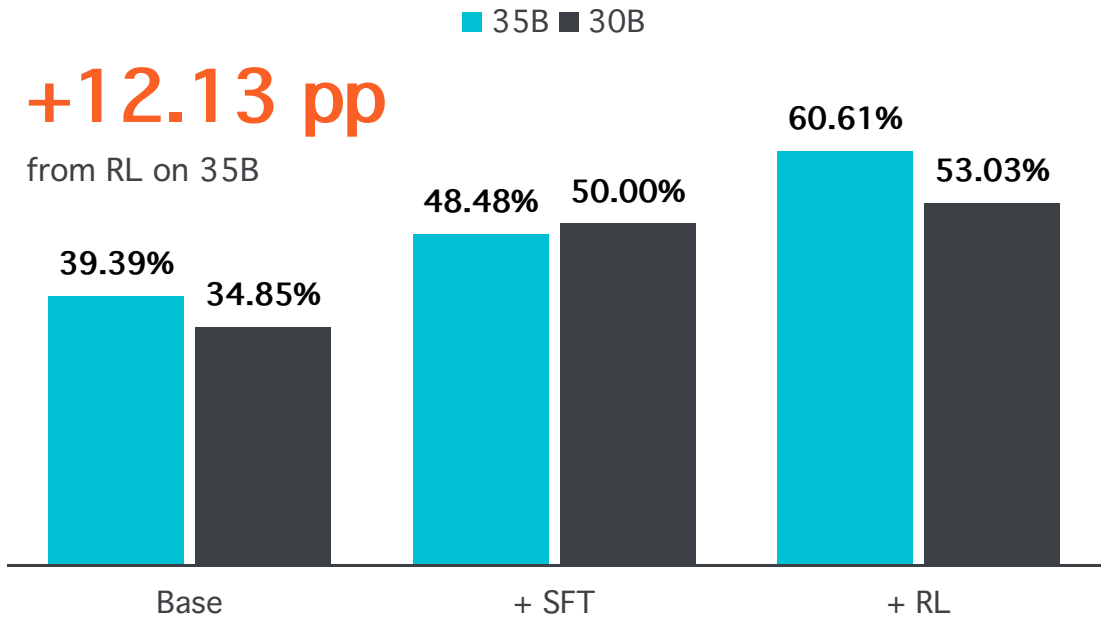
Consume experiments as they finish.

Source: Yang et al., Frontis-MA1, arXiv:2607.28568, Fig. 6 and Sec. 4.2 (2026).

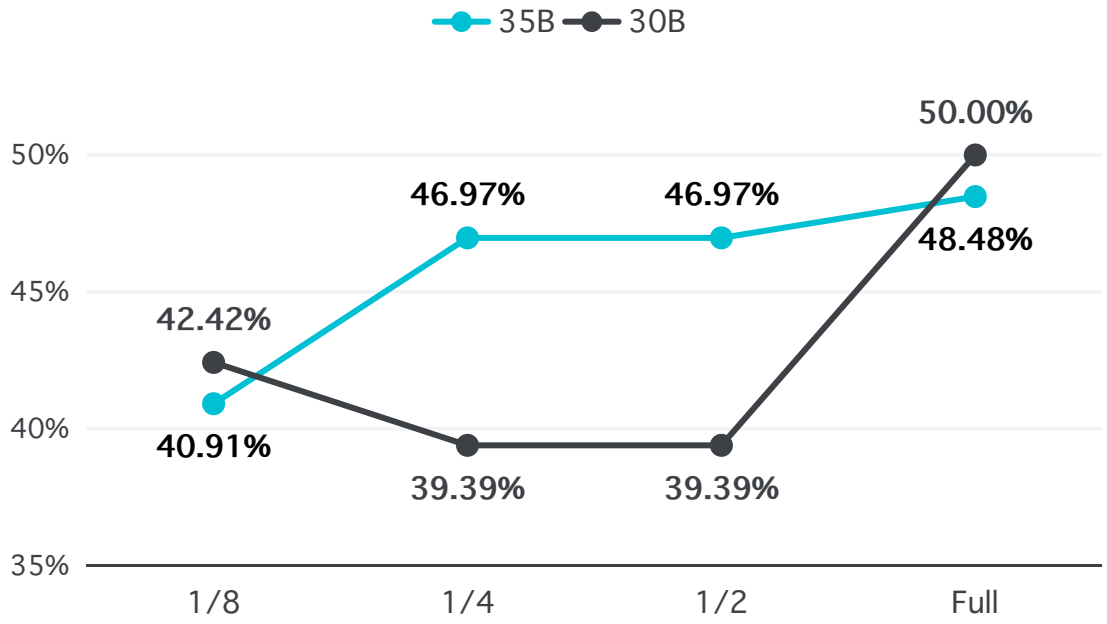
Training gains come from SFT, RL and task coverage

MLE-Bench Lite, fixed search harness. Medal Average = share of tasks earning any medal, averaged over three runs.

TRAINING STAGES

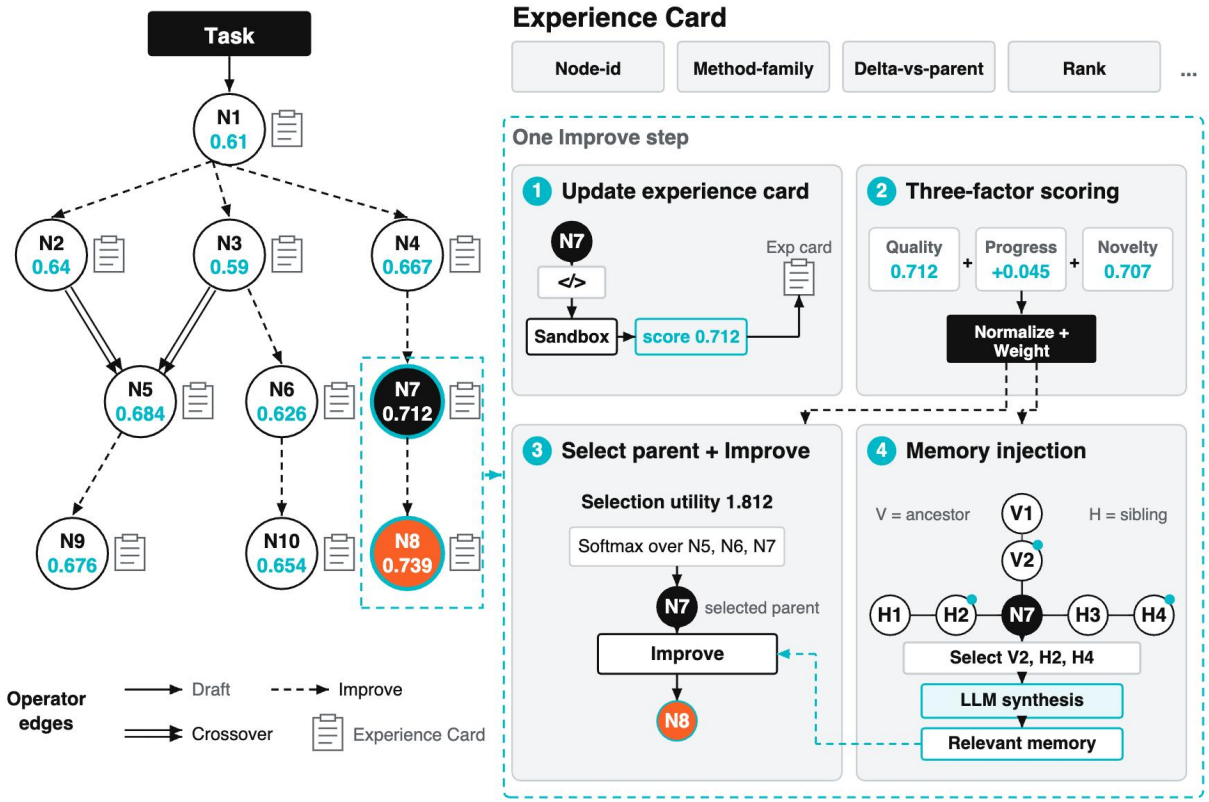


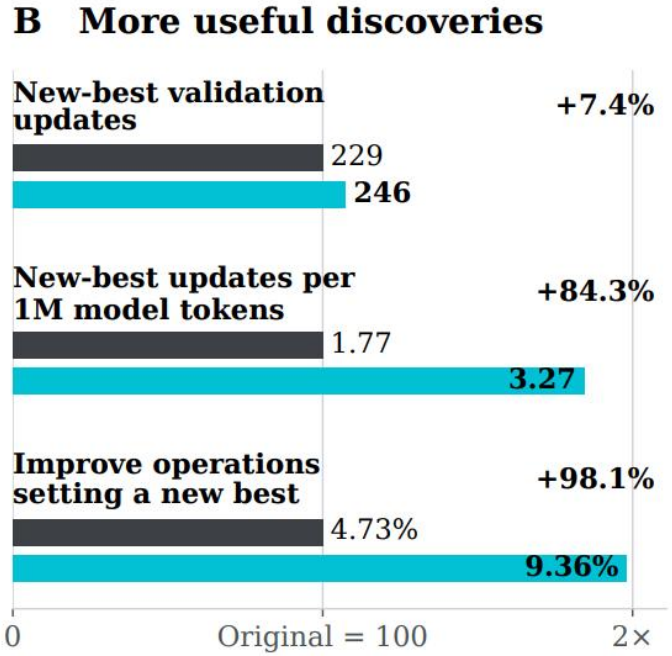
SFT TASK COVERAGE · FIXED STEPS AND BATCH



OpenMLE-Evo makes experience useful to search

Same model and budget: 1.77 → 3.27 new-best updates per million model tokens.





−41.7% model tokens

129.3M → 75.3M, matched task-runs

Source: Frontis-MA1, Figs. 9 and 16. Matched checkpoint, seeds and execution budgets.

Revisiting an earlier branch improves protein predictions

NatureBench Lite case study: Frontis-MA1 reaches g = +0.1161 on a held-out protein task.

PROTEIN BIOLOGY

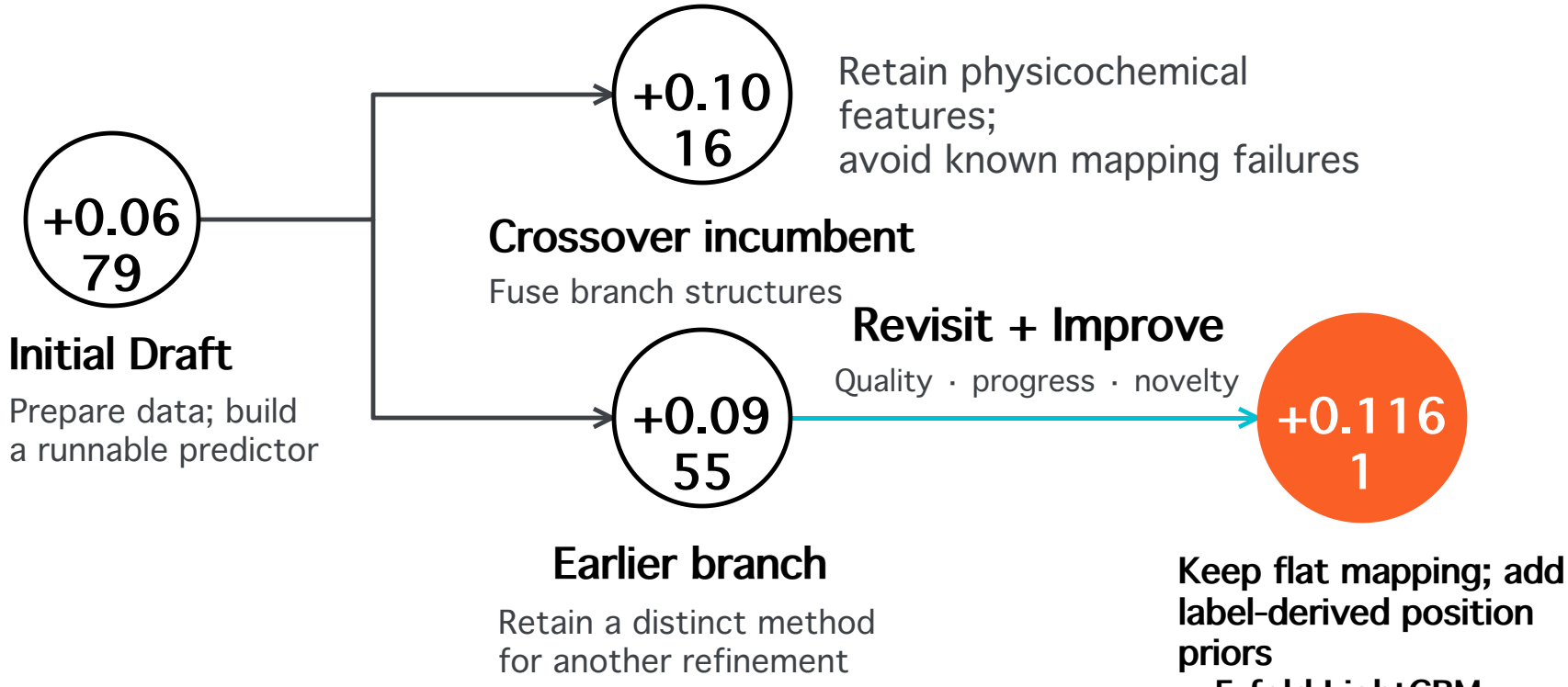
Protein variant effect prediction

11 protein-assay instances
4 hours per task
No network access

BASE MODEL

g = +0.0243

SEARCH REVISITS A NON-LEADING BRANCH

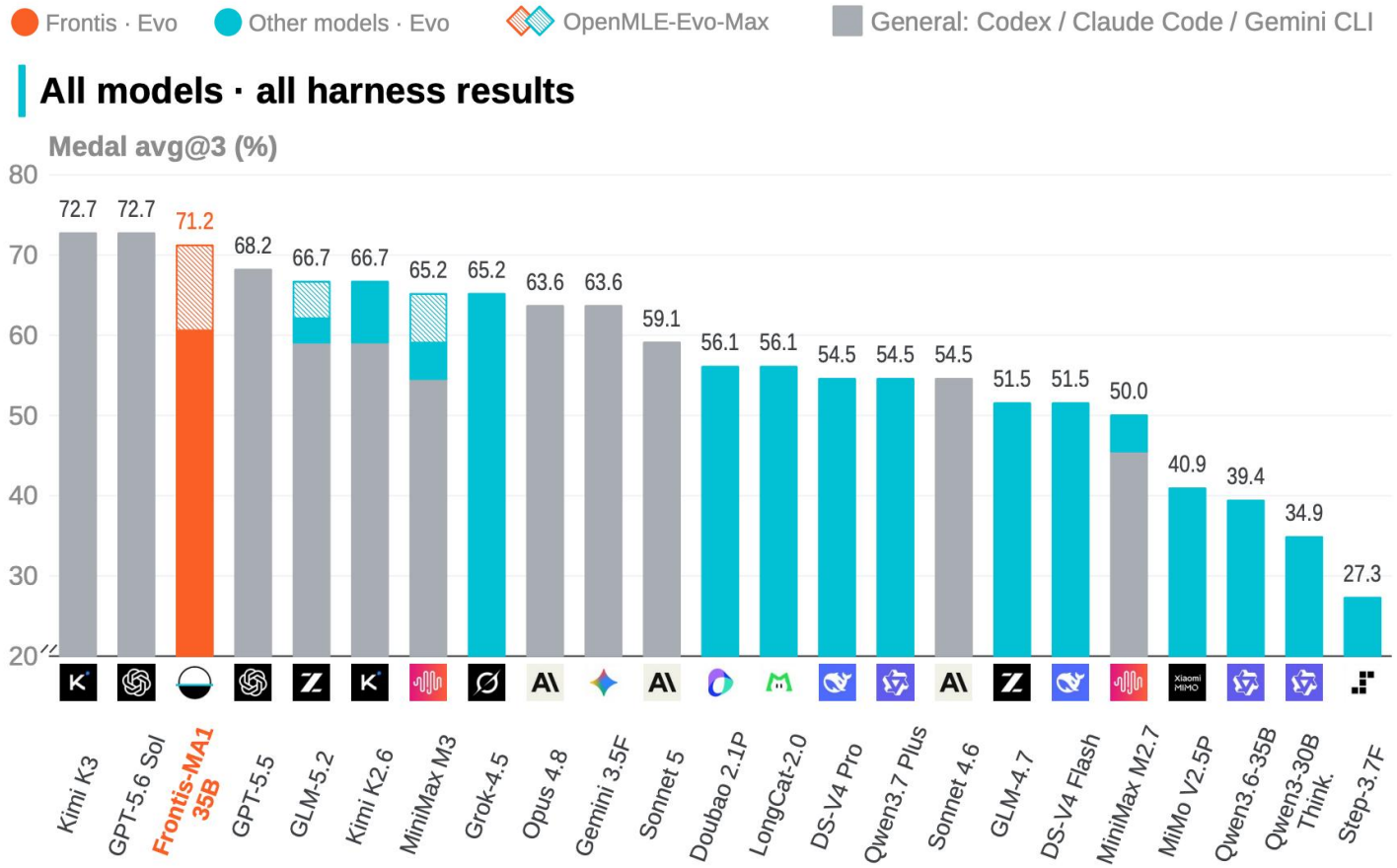


g is the gap to the paper reference, not accuracy. One task-level trace; intermediate nodes omitted.

Source: Frontis-MA1, pp. 28–29. Held-out task; no hidden test labels or paper solution.

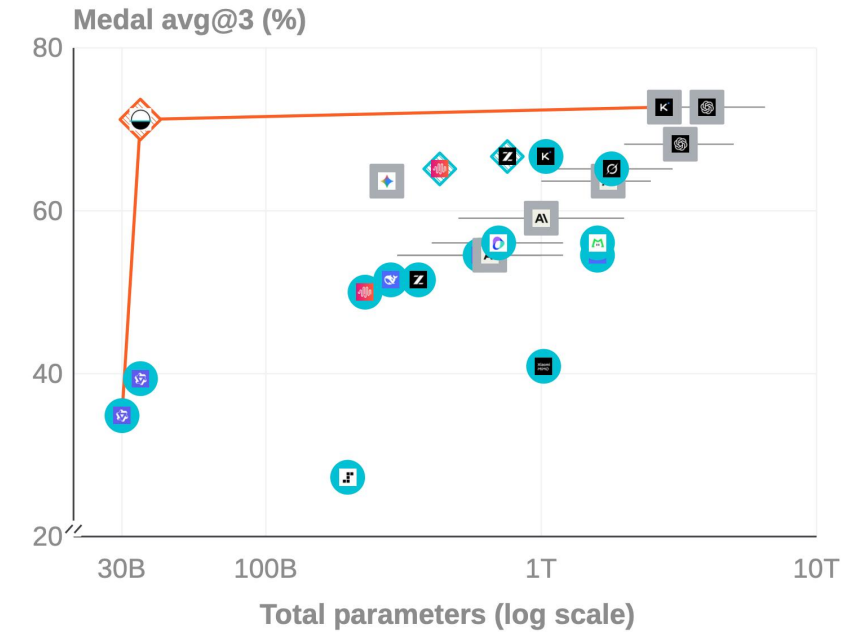
Frontis-MA1 delivers strong results at 35B parameters

Paper Fig. 1: MLE-Bench Lite results across models and harnesses, plus the model-size comparison.



71.21%

35B parameters · OpenMLE-Evo-Max



Source: Frontis-MA1, Fig. 1 / Table 1. MLE-Bench Lite, 22 tasks, three runs; 12h per task.

OpenRSI: every layer of the loop is open

Start with one local task, serve a model, and inspect a real search trace before scaling up.

OpenMLE-Gym
Task packages + reconstruction scripts

OpenMLE Sandbox
Execute code, collect structured feedback

OpenMLE-ERL
SFT and RL training code

Frontis-MA1
30B / 35B weights + GGUF builds

OpenMLE-Evo
Experience-guided search runtime

Supervised traces
26,259 execution-grounded examples

github.com/FrontisAI/OpenRSI

1,415 complete task packages + 4,343 reconstruction recipes. Code, data and weights carry their own licenses.

A trained improver can keep evolving

OpenMLE trains revision skills into weights and composes them through search.

WHAT TRAINING GIVES US

Executable task supply
OpenMLE-Gym and the sandbox

A trained revision model
SFT and RL on execution feedback

Experience-guided search
OpenMLE-Evo composes the revisions

NEXT: SELF-EVOLUTION OF THE IMPROVER

Can the improver revise its own strategy?

MetaEvoForge
Frozen model weights. Persistent policy updates. Fresh task pools.

Improvement experience now updates the improver's own policy.

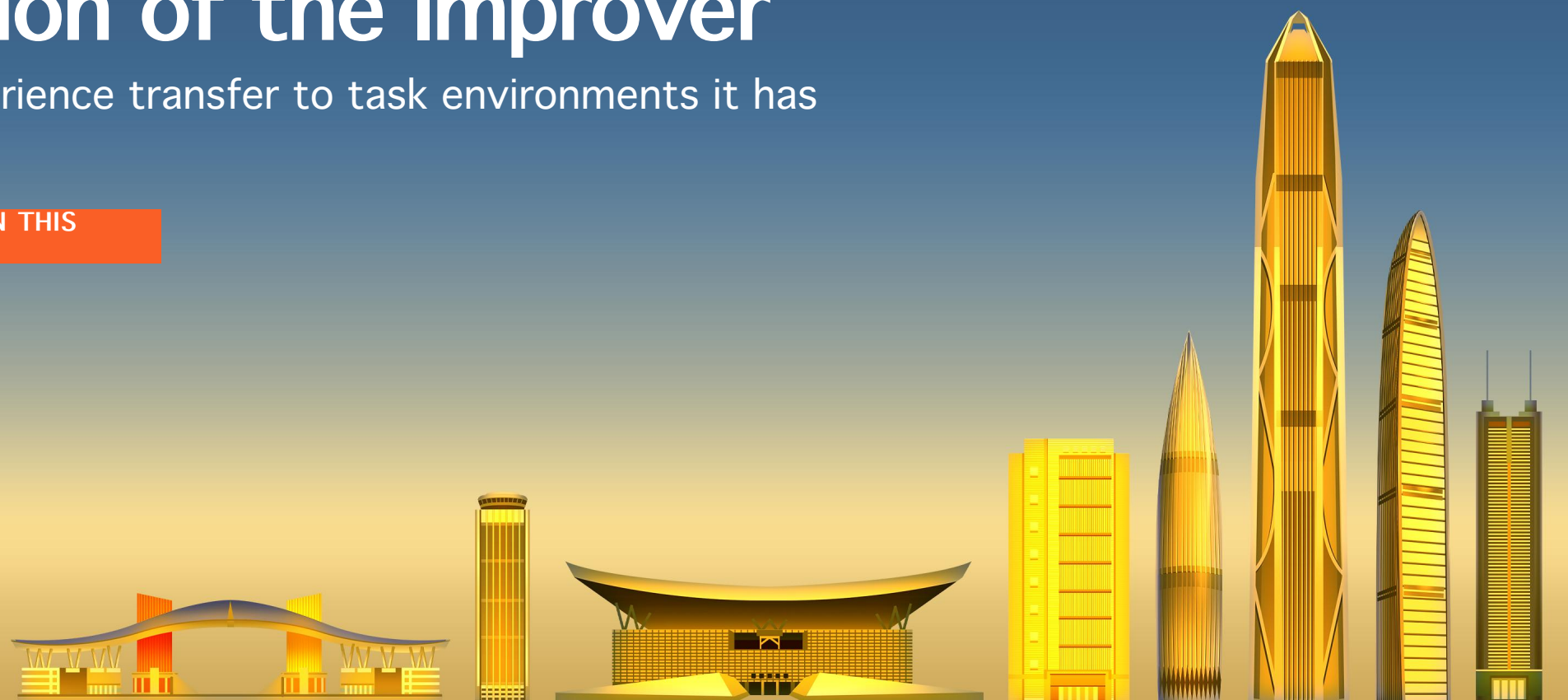
04

Self-Evolution of the Improver

Can improvement experience transfer to task environments it has never seen?

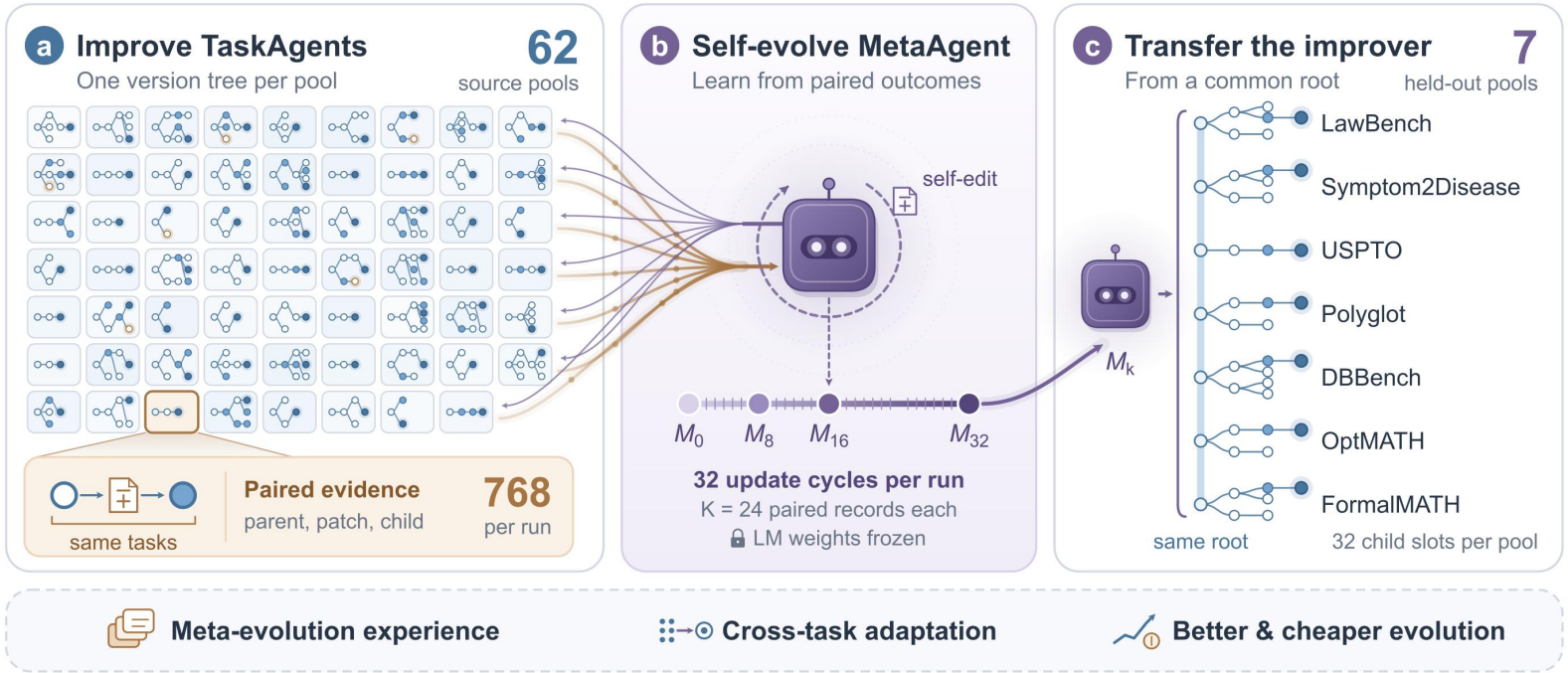
METAEVOFORGE

NEW IN THIS TALK



MetaEvoForge: an improver that learns across tasks

TaskAgents solve tasks. A separate MetaAgent edits them, learns from paired outcomes, and is tested on unseen pools.



WHY A SEPARATE IMPROVER?

A better agent is not necessarily a better improver.

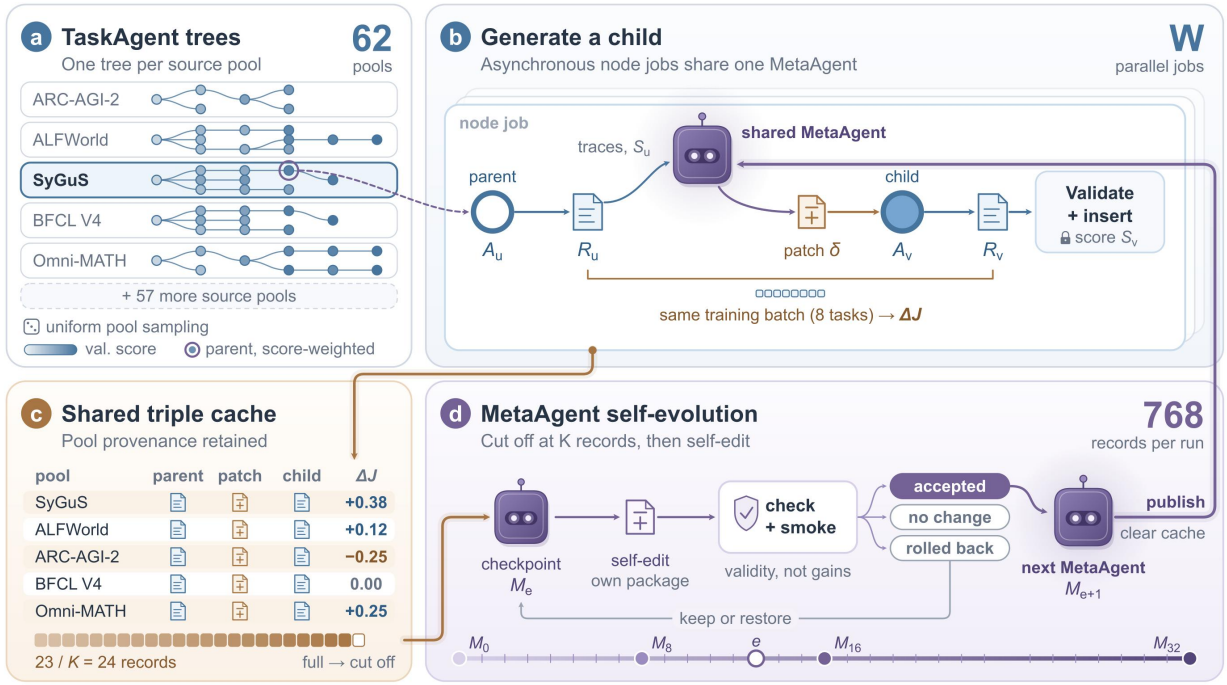
- **RQ1** Does a trained MetaAgent transfer to unseen task pools?
- **RQ2** What does adaptation cost versus external improvers?
- **RQ3** Which learned behaviors help, and where does transfer fail?

NEW IN THIS TALK

Source: MetaEvoForge, Fig. 1.

Each edit becomes a parent–patch–child record

Within a pool, TaskAgents evolve as a version tree. Across pools, one shared MetaAgent learns from the records.



1 Propose

Sample a pool and a score-weighted parent. The MetaAgent proposes a scoped patch δ to its package.

2 Pair

Run parent and child on the same 8 tasks: ΔJ = J(child) − J(parent). Validation returns a score, never answers.

3 Self-update

After K = 24 records the MetaAgent edits its own package. Checks decide accept, no change or rollback.

4 Repeat

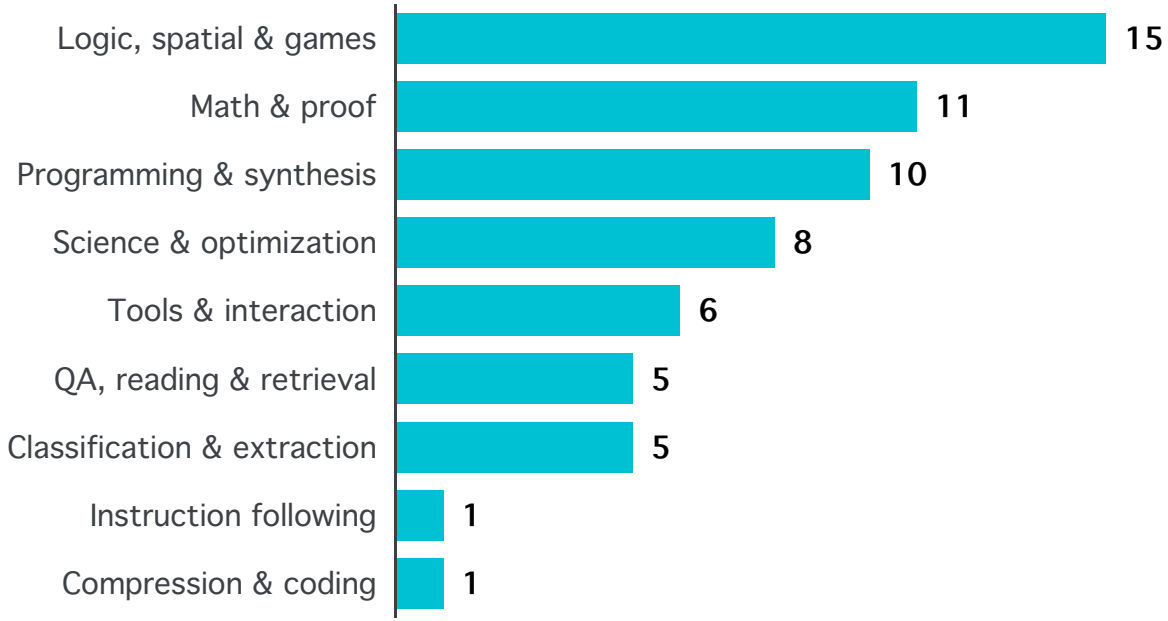
32 cycles → 768 records per run. Checkpoints M_0, M_8, M_{16}, M_{32}.

Source: MetaEvoForge, Fig. 2 and Algorithm 1.

Trained on 62 task pools, tested on 7 unseen ones

Pools are disjoint by identifier; target-pool adaptation packages never enter MetaAgent training.

62 SOURCE POOLS BY TASK FAMILY



62 source pools

8,315 training + 1,592 validation packages

7 held-out pools

742 adaptation · 356 validation · 1,078 final-test packages

HELD-OUT POOLS

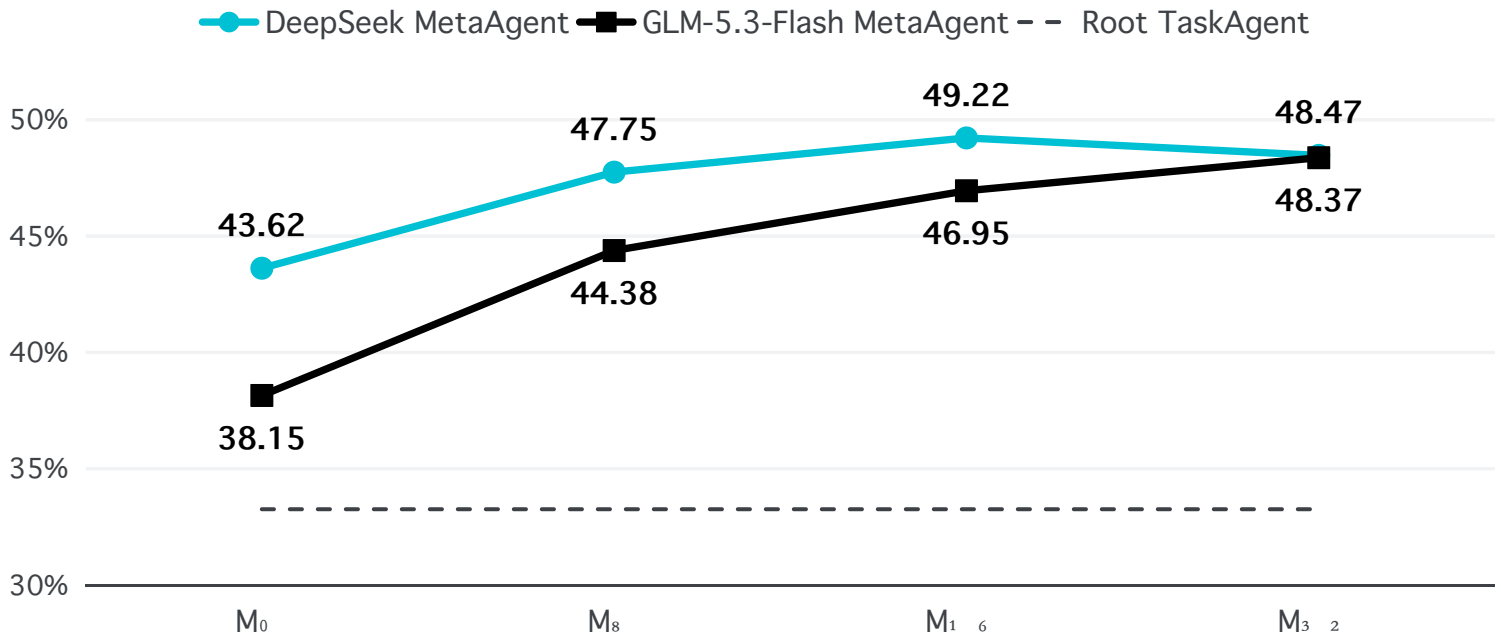
- LawBench
- Symptom2Disease
- USPTO
- Polyglot
- DBBench
- OptMATH
- FormalMATH-Lite

Source: MetaEvoForge, Sec. 5, Fig. 3b and Tables 3–4.

Trained improvers transfer to unseen task pools

Seven held-out pools, same root TaskAgent, 32 child slots each. Final-test macro score of the validation-selected agent.

MACRO SCORE (%) BY METAAGENT CHECKPOINT



+5.60 pp

DeepSeek MetaAgent

M_{16} vs its own M_0 : 43.62 → 49.22

+10.22 pp

GLM-5.3-Flash MetaAgent

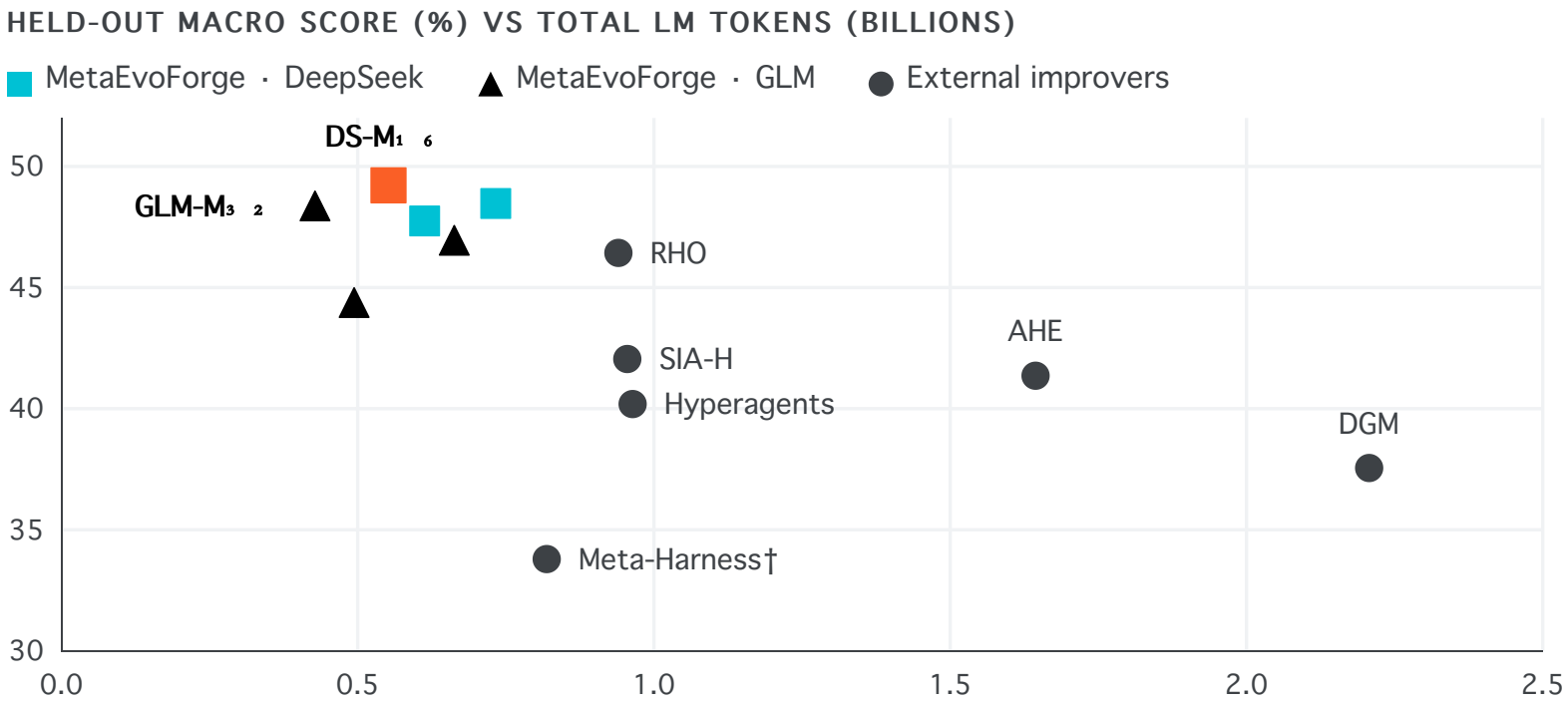
M_{32} vs M_0 : 38.15 → 48.37, better on all 7 pools

Every trained checkpoint beats its own initialization.

Source: MetaEvoForge, Table 1.

Better and cheaper than six external improvers

Same root, same TaskAgent model, harness and sandbox for every method. Up and to the left is better.



49.22%

DS-M16 macro score

vs 46.43% for RHO, the strongest external improver

−41%

LM tokens

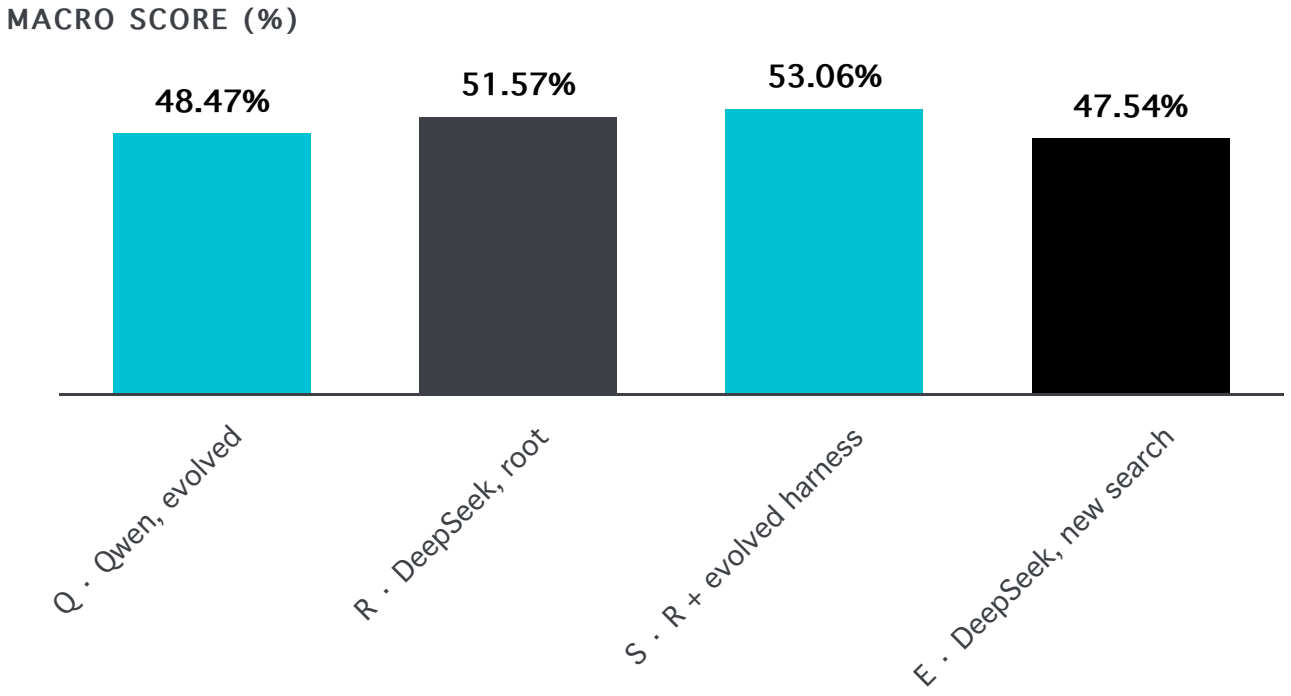
0.55B vs RHO's 0.94B

Single baselines still lead on some pools: RHO on LawBench, Hyperagents on Symptom2Disease.

Source: MetaEvoForge, Fig. 4.

A useful harness transfers; the strategy may not

Swap the TaskAgent's base model and compare four conditions on the seven held-out pools.



+1.49
S − R

Swapping only the base model keeps useful behavior: the evolved harness beats the DeepSeek root on 4 of 7 pools.

−5.52
E − S

A fresh search with the same MetaAgent falls below the swap: the strategy is partly tuned to Qwen-based evolution.

−3.10
Q − R

An evolved agent on the smaller Qwen model gets within 3.1 points of the unevolved DeepSeek agent.

Source: MetaEvoForge, Table 2.

What the improver learned, and where it misfires

768 annotated proposals across 32 self-update cycles, plus two regressions worth remembering.

WHERE EDITS LAND · 768 PROPOSALS



● Skills only 76.6% ● Harness profile 11.8% ● Plugins 8.2% ○ Mixed 3.4%

HOW PROPOSALS SHIFT · CYCLES 1–8 → 25–32

Contract and runtime fixes	31.8% →	**21.4%**
Pool-specific adaptation	55.7% →	**65.1%**
Transferable reasoning	peaks 20.3% →	**13.5%**

Accepted self-updates rewrite the MetaAgent's own policy: revisit guidance that did not transfer, and skip stalled runs before crediting a patch.

WHERE IT MISFIRES

Symptom2Disease · MetaEvoForge

81.60 → 73.58

M_0 wrote a benchmark guide; later cycles piled on exception rules.

DBBench · AHE baseline

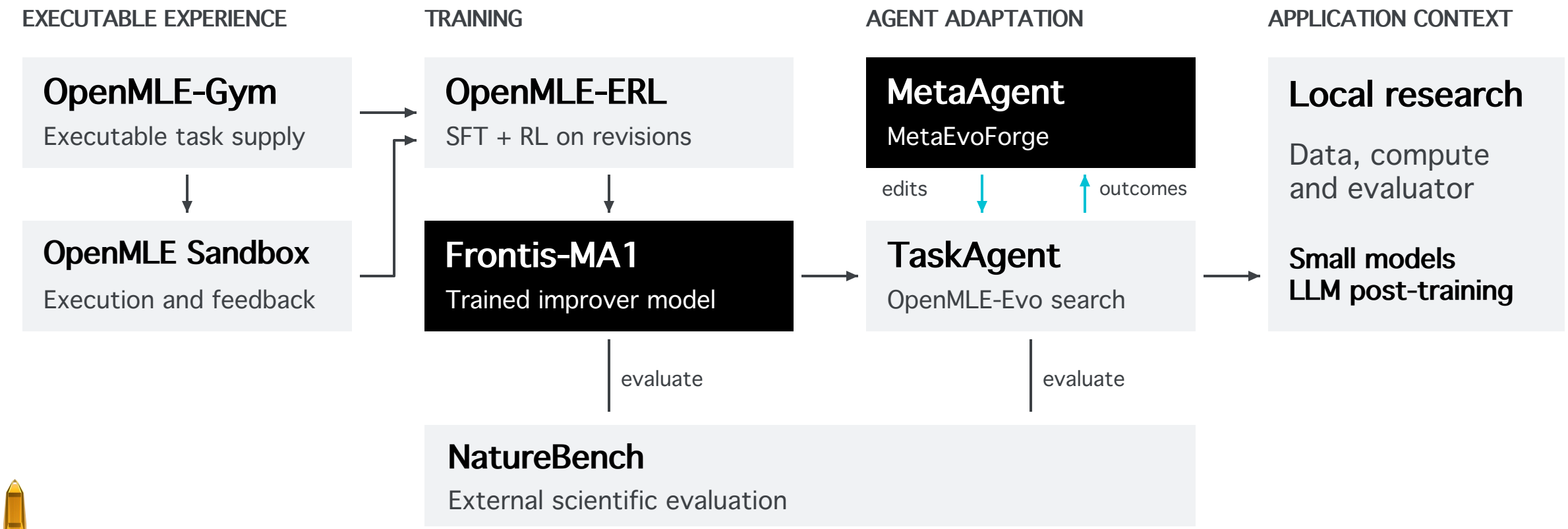
A new column checker raised training accuracy, then issued a false warning on validation and the agent swapped a correct answer for a wrong one.

> **More retained rules can hurt. Self-updates need tests for reuse, revision and deletion.**

Source: MetaEvoForge, Secs. 6.1, 7.1 and 7.3.

Our landscape for AI that improves AI

Executable experience, a trained improver and an agent that evolves its own improvement policy.



Four lessons for building AI that improves AI

Observable progress → better experiments → transferable knowledge → co-evolution

01 **Feedback determines what an AI can learn to improve.**

Build environments where meaningful progress becomes observable.

02 **Experience should improve the allocation of future experiments.**

Learn which change to try next, under which conditions, and at what cost.

03 **The enduring asset is transferable improvement knowledge.**

Carry improvement strategies across tasks, agents, and model generations.

04 **Sustained progress requires co-evolution of the learning loop.**

Advance the model, the improvement strategy, and the task environment together.

GET INVOLVED

- **Run an OpenMLE loop**
  in your own domain · github.com/FrontisAI/OpenRSI
- **Evaluate on NatureBench**
  via the Harbor adapter or NatureBench-25
- **Stress-test the MetaAgent**
  framework when it ships

GOSIM

GOSIM Shenzhen 2026

Thank you

Che Jiang
Research Scientist at FrontisAI
Doctoral Researcher at Tsinghua University

WITH GRATITUDE TO OUR COLLABORATORS


Che Jiang


Kaiyan Zhang

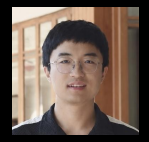

Junlin Yang


Weizhi Wang


Yu Fu


Yuru Wang


Lejun Cheng


Kaikai Zhao


Jincheng Zhong


Yuxin Zuo


Tianwei Luo


Can Ren


Kai Tian


Yuchong Wang


Hongyi Liu

WE ARE HIRING

GitHub

Hugging Face

Website

WeCha
t · TsinghuaC3I